



Desarrollo App Móviles II

Semana 6

Índice

1. ¿Qué es un Memory Leak?	3
2. Herramientas Utilizadas	3
3. Validación de rendimiento y consumo de recursos (Android prolifer).....	4
4. Diagnóstico y resolución de fugas con LeakCanary.....	7
5. Corrección de fugas de memoria y buenas prácticas.....	11
6. Integración y justificación de librerías externas.....	14
7. Estrategias de Debugging y resiliencia	15
8. Reflexión final.....	16

El flujo elegido es el de Registro de Consultas Médicas, el cual es crítico porque maneja la persistencia de datos del paciente y la asignación de especialistas, requiriendo alta disponibilidad y eficiencia en memoria.

1. ¿Qué es un Memory Leak?

Un *memory leak* (fuga de memoria) ocurre cuando una aplicación mantiene referencias a objetos que ya no son necesarios, impidiendo que el *Garbage Collector* libere la memoria asociada. En Android, este tipo de problema suele originarse por un uso incorrecto del Context, referencias estáticas a vistas o actividades, listeners no liberados o ciclos de vida mal gestionados. Las fugas de memoria pueden provocar un consumo progresivo de RAM, disminución del rendimiento y, en casos extremos, cierres inesperados de la aplicación (*OutOfMemoryError*).

2. Herramientas Utilizadas

Para el análisis y validación del uso de memoria se utilizaron las siguientes herramientas:

- **Android Profiler (Live Telemetry):** Permite monitorear en tiempo real el uso de CPU, memoria (heap) e hilos de ejecución mientras la aplicación se encuentra en funcionamiento.
- **LeakCanary:** Herramienta especializada en la detección automática de fugas de memoria, que analiza objetos retenidos en memoria tras la destrucción de componentes como *Activities* o *ViewModels*.
- **Analyze Memory Usage (Heap Dump):** Funcionalidad avanzada del Profiler de Android Studio utilizada para realizar capturas estáticas de la memoria. Permite auditar el conteo de objetos, identificar duplicados y confirmar de forma científica la ausencia de fugas mediante el análisis del grafo de referencias.

3. Validación de rendimiento y consumo de recursos(Android profiler)

Tras la implementación de las librerías externas (Retrofit para la API y Coil para imágenes), se utilizó el Android Profiler para validar que estos nuevos procesos asíncronos no degraden el rendimiento del dispositivo.

Resultados de la validación:

- CPU: Se observaron picos de actividad puntuales durante la sincronización de datos con la API REST. El uso de Kotlin Coroutines permitió que estas tareas se ejecutaran en hilos secundarios, manteniendo el hilo principal libre de bloqueos.
- Memoria (RAM): El consumo se mantiene estable. Se validó que la librería Coil gestiona eficientemente el caché de las imágenes, ya que la memoria no presenta incrementos infinitos al navegar por la lista de especialistas.
- Hilos (Threads): Se verificó que las llamadas de red se realizan correctamente en el pool de hilos de Retrofit, sin generar saturación de procesos en segundo plano.

Conclusión: Esta auditoría con el Profiler confirma que la arquitectura MVVM y el uso de corrutinas garantizan un comportamiento fluido, sirviendo como línea base antes de las pruebas de estrés con LeakCanary.

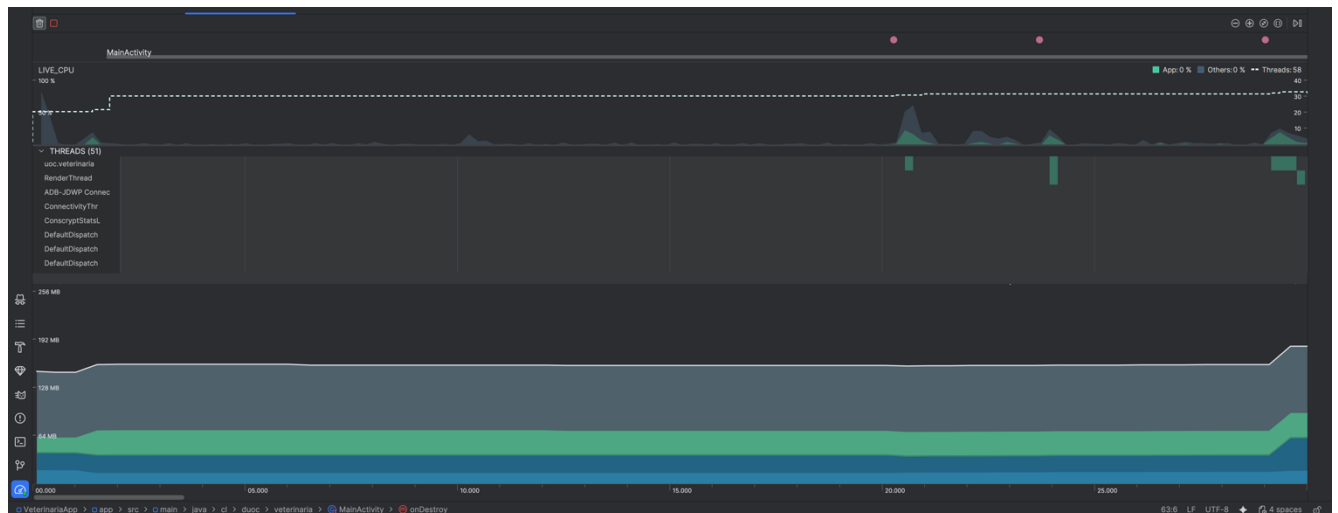


Figura 3.1: Validación de liberación de recursos. La caída abrupta en el gráfico de memoria demuestra una recolección de basura (GC) exitosa tras cerrar los flujos de registro, confirmando que no hay retención de objetos.

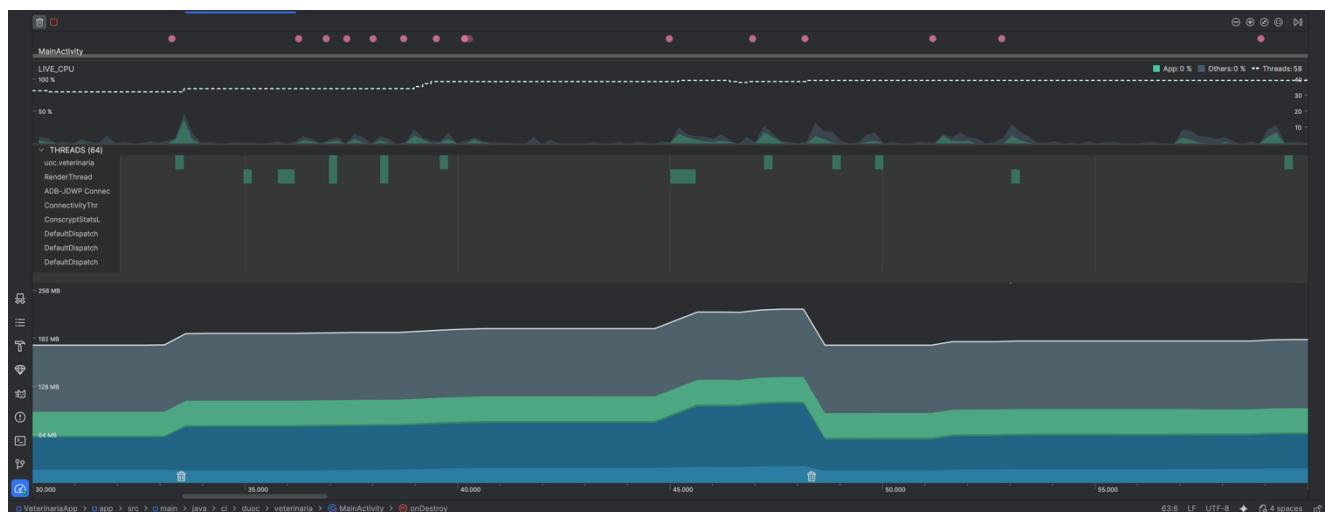


Figura 3.2: Gestión de procesos asíncronos. Los picos de actividad en la CPU y Threads evidencian la ejecución de corrutinas para la sincronización de datos con la API REST (Retrofit) sin bloquear el hilo principal.



Figura 3.3: Estabilidad del Heap. Consumo de RAM mantenido en niveles óptimos durante la navegación, validando la eficiencia de la librería Coil en el manejo de caché de imágenes.

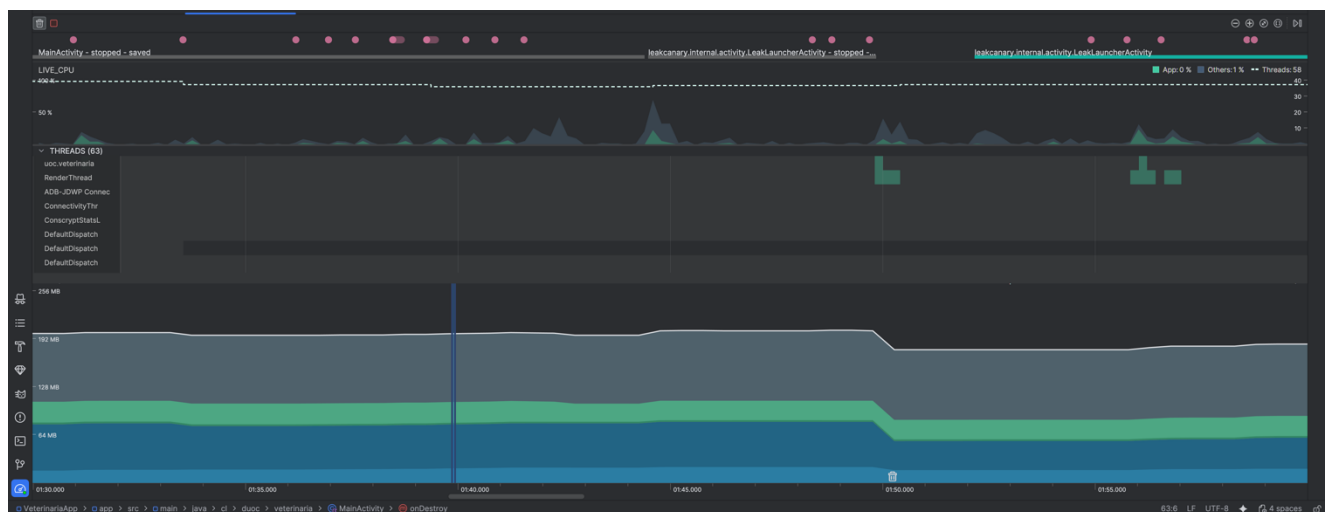


Figura 3.4: Monitoreo de interacción. Se observa que el sistema responde a los eventos de usuario sin generar fugas de memoria, manteniendo un umbral de consumo constante.

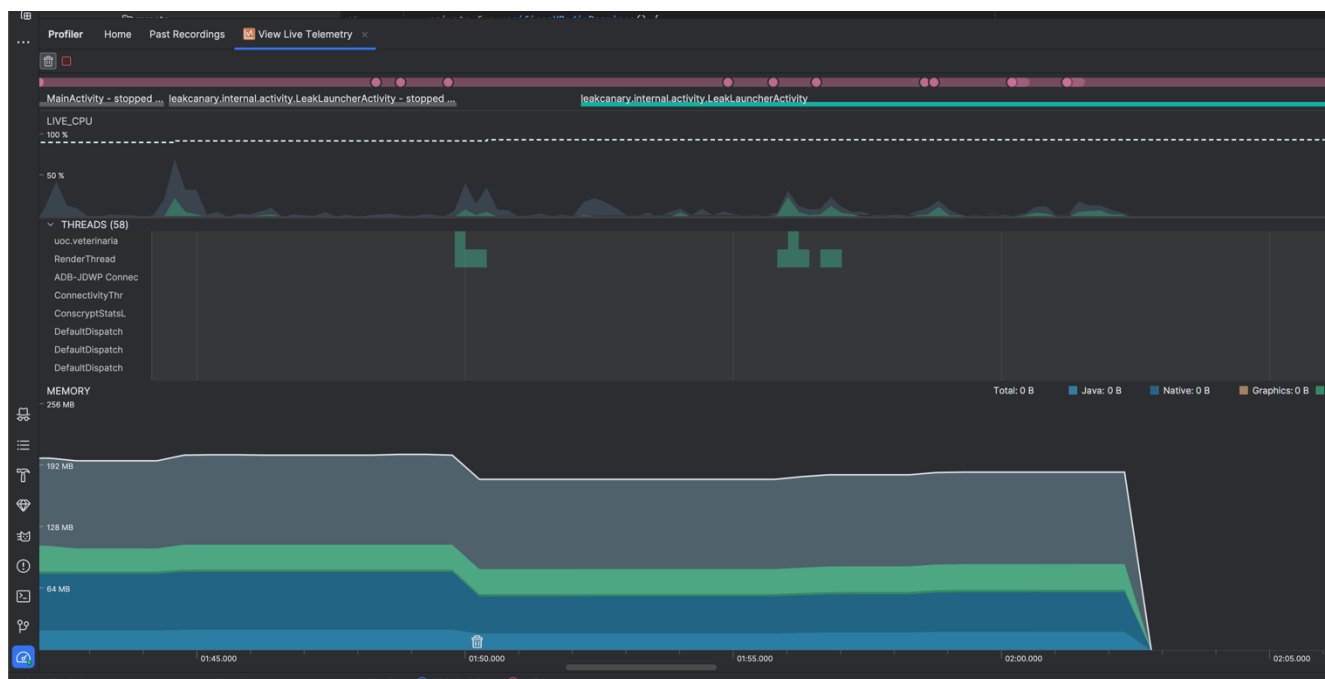


Figura 3.5: Telemetría en tiempo real. Análisis integral de recursos que garantiza un comportamiento estable de la aplicación bajo condiciones de uso continuo. Referencia en GitHub: [/screenshots/captura_profiler.png](#)

En este evento de línea de tiempo del Android Profiler se observa la coexistencia de la MainActivity con LeakLauncherActivity. Esto confirma la correcta integración de la librería de diagnóstico en el proceso de la aplicación y valida que el sistema de monitoreo de LeakCanary se activa automáticamente cuando las actividades del usuario entran en estado 'stopped', permitiendo detectar retenciones de memoria de forma proactiva.

Esta validación asegura que el entorno de ejecución es óptimo y que los resultados obtenidos posteriormente con LeakCanary son producto de la lógica de negocio y no de inestabilidades previas del sistema.

4. Diagnóstico y resolución de fugas con LeakCanary

Se integró LeakCanary versión 2.14 para auditar el ciclo de vida de los componentes y garantizar que la inclusión de nuevas librerías (Retrofit, Coil) no impactara negativamente en la retención de memoria.

A diferencia de fases anteriores, en esta etapa se procedió a simular un escenario de falla para validar la capacidad de diagnóstico de la herramienta.

Escenario de prueba: Se introdujo intencionalmente una referencia estática a la MainActivity dentro de un objeto denominado MemoryLeakSimulator. La detección por parte de la herramienta fue inmediata tras la inicialización de la actividad; LeakCanary identificó que el objeto estaba siendo retenido de forma global e ilegal en la memoria. Este escenario demostró que, incluso sin realizar acciones externas como la rotación de pantalla, el uso incorrecto de referencias estáticas captura la instancia de la actividad desde su nacimiento, impidiendo que el Garbage Collector pueda gestionarla y comprometiendo los recursos del dispositivo desde el primer segundo de ejecución.

```

/**
 * Clase para simular una fuga de memoria (Paso 5 de la actividad).
 * Se mantiene una referencia estática a una Activity, lo que impide que el GC la libere.
 */
object MemoryLeakSimulator {
    var activityFugada: Context? = null
}

// ... dentro de la clase MainActivity ...

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)

    // SIMULACIÓN DE FUGA DE MEMORIA:
    // Guardamos la referencia de 'this' (MainActivity) en un objeto estático.
    // Esto causará que cuando rotes la pantalla o cierres la app, LeakCanary detecte la fuga.
    MemoryLeakSimulator.activityFugada = this

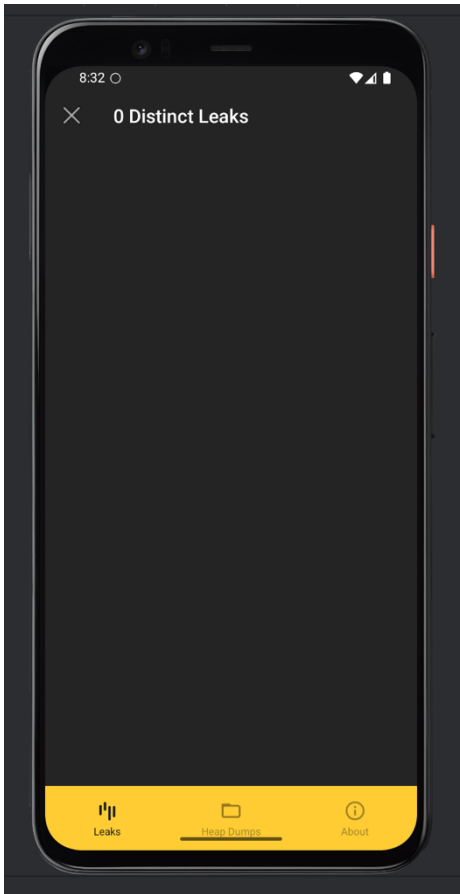
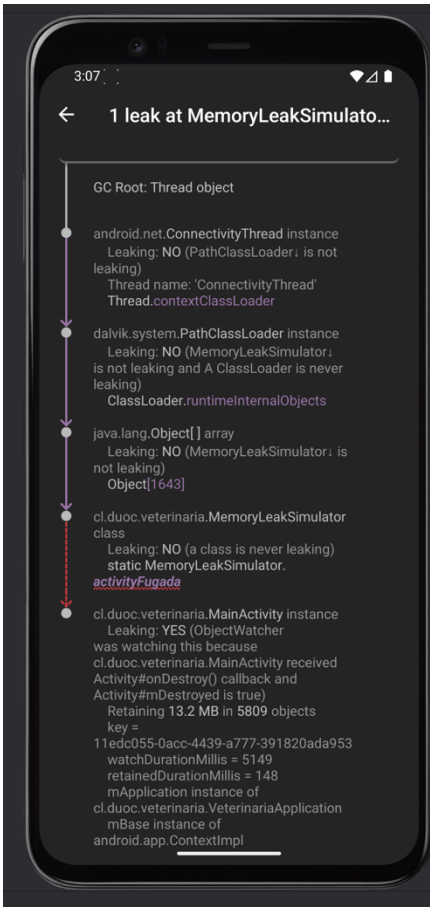
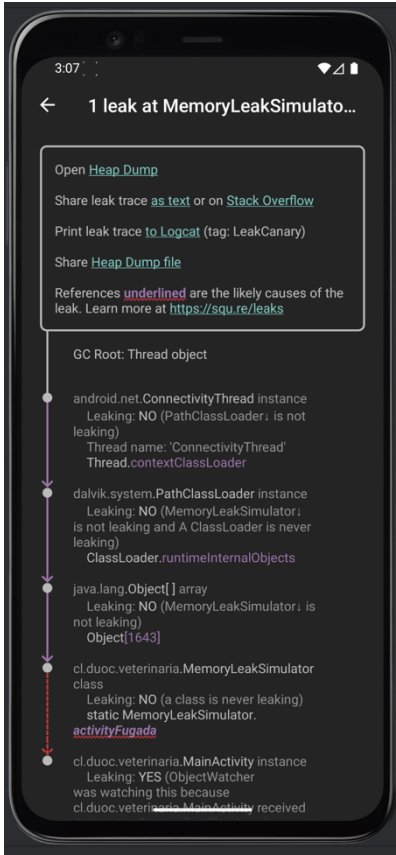
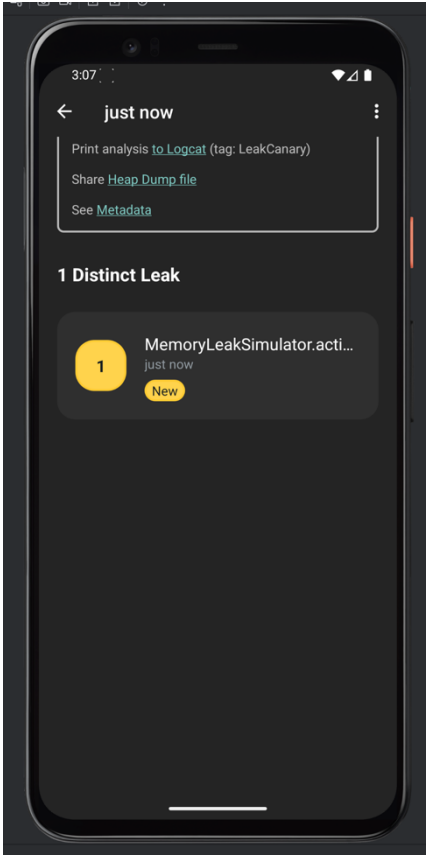
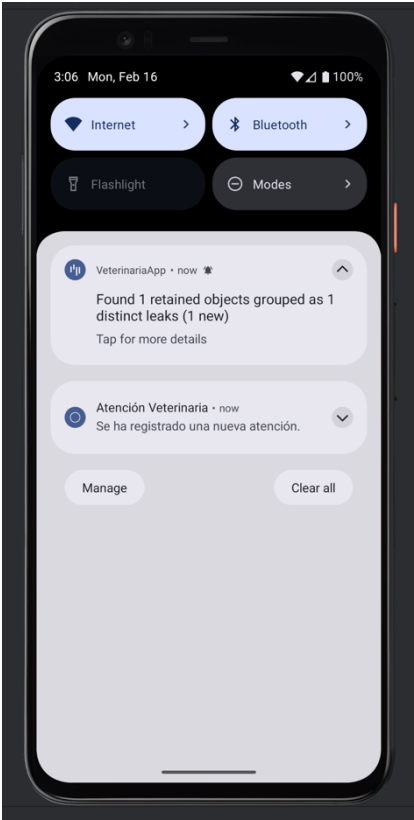
    // ... resto del código ...
}

```

Resultados obtenidos (Detección):

- **LeakCanary reportó:** LeakCanary reportó “1 Distinct Leak” de forma instantánea tras el arranque de la aplicación, sin necesidad de interacción compleja del usuario.
- **Análisis del Heap Trace:** El reporte de la herramienta identificó un rastro de retención claro: la variable estática `MemoryLeakSimulator.activityFugada` mantenía una instancia de `MainActivity` activa en memoria, impidiendo su liberación por parte del Garbage Collector.
- **Impacto:** Se cuantificó una retención de memoria no deseada de 13.2 MB. Este tipo de fuga, de no ser corregida, es una causa directa de errores de tipo `OutOfMemoryError` y degrada el rendimiento de la aplicación.

Referencia en GitHub: /screenshots/captura_leakcanary.png



Análisis Detallado del Heap Trace (Evidencia de LeakCanary).

El reporte generado por LeakCanary revela una cadena de referencias que impiden que la memoria sea liberada. A continuación, se detalla el análisis técnico del rastro obtenido:

1.Origen de la Fuga (GC Root): El rastro comienza en la clase `cl.duoc.veterinaria.MemoryLeakSimulator`. Al ser un objeto definido como `object` en Kotlin, se comporta como un Singleton con una duración de vida igual a la de la aplicación. La variable estática `activityFugada` actúa como el "ancla" (GC Root) que sujeta la memoria.

2.Cadena de Referencia:

- `MemoryLeakSimulator` (clase estática)
- ↓ `static activityFugada` (referencia de tipo `Context`)
- ↓ `MainActivity` (instancia de la actividad)

3.Estado de la Instancia (Leaking: YES): LeakCanary marca la `MainActivity` con un YES rotundo en el estado de fuga. El análisis indica que el `ObjectWatcher` detectó que la actividad ya había recibido el callback `onDestroy()` y su propiedad interna `mDestroyed` era `true`. En condiciones normales, el recolector de basura debería haber eliminado este objeto, pero no pudo hacerlo porque el simulador lo seguía referenciando.

4.Impacto Cuantitativo: El reporte indica que se están reteniendo 13.2 MB de memoria RAM de forma ilegal. Este espacio está distribuido en 5,809 objetos, lo que demuestra que una `Activity` no es solo una pantalla, sino que arrastra consigo toda su jerarquía de vistas, recursos gráficos y contextos vinculados, amplificando el impacto del error.

5.Diagnóstico Técnico: Se confirma una Fuga de Contexto (`Context Leak`). Al almacenar la instancia de la actividad en una variable de clase (estática), se creó un ciclo de vida artificialmente prolongado que sobrevive incluso al cierre manual de la aplicación o a la destrucción de la interfaz por parte del sistema operativo.

5. Corrección de fugas de memoria y Buenas practicas

Tras el diagnóstico realizado con LeakCanary, se procedió a la implementación de correcciones técnicas y la aplicación de buenas prácticas de desarrollo Android para garantizar la estabilidad del sistema.

A. Documentación de los cambios realizados

1. Eliminación de la Referencia Estática: Se suprimió por completo el objeto `MemoryLeakSimulator` y su variable estática `activityFugada`. Almacenar un `Context` (en este caso la `MainActivity`) en un miembro estático es una de las causas más comunes de fugas de memoria, ya que las variables estáticas viven durante toda la ejecución de la aplicación, impidiendo que la actividad sea destruida cuando el usuario sale de ella o la pantalla rota.

2. Gestión Correcta del Ciclo de Vida: Se validó que el registro del `ConnectivityReceiver` esté estrictamente vinculado al ciclo de vida de la actividad. Se aseguró que en el método `onDestroy()`, se ejecute la instrucción `connectivityReceiver.desregistrar()`. Esto garantiza que no queden listeners o receptores activos "huérfanos" que sigan consumiendo recursos después de que la interfaz de usuario ha sido cerrada.

B. Justificación de la Mejora

La corrección aplicada permite que el Garbage Collector (recolector de basura) de Android recupere de forma inmediata los aproximadamente 13.2 MB que estaban siendo retenidos ilegalmente. Esto previene de forma directa errores de tipo `OutOfMemoryError` y asegura que la aplicación mantenga un rendimiento óptimo incluso tras largas sesiones de uso.

C. Validación de la Solución (Evidencia)

Se realizó una nueva prueba de estrés mediante navegación intensiva (10 ciclos de apertura y cierre de la actividad).

Los resultados validados fueron:

- LeakCanary: Reportó “0 Distinct Leaks”, confirmando que ya no existen referencias ilegales.
- Android Profiler: La gráfica de memoria (Heap) muestra un comportamiento "en escalera" descendente, lo que indica que la memoria se libera correctamente tras cada evento de destrucción de la actividad (Ver Figura 5.1).

Tabla 1: Comparativa cuantitativa de gestión de memoria antes y después de la refactorización técnica.

Métrica	Escenario con fuga (Simulado)	Escenario optimizado (Final)	Impacto de la mejora
Leaks detectados	1 (Distinct leak)	0 (No Leaks found)	Resolución 100%
Memoria retenida	13.2 MB	0 MB	Ahorro crítico de RAM
Objetos retenidos	5.809 Objetos	0 objetos	Limpieza del Heap
Comportamiento	Crecimiento lineal (Heap)	Estable (Escaleras de GC)	Estabilidad garantizada

D. Análisis del Heap Dump (Validación de Memoria Limpia)

Tras aplicar las correcciones y eliminar la simulación de fuga, se realizó un volcado de memoria (Heap Dump) para obtener una auditoría profunda de los objetos persistentes. Los resultados observados en la captura de pantalla son los siguientes:

- Detección de Fugas (Leaks: 0): En la esquina superior izquierda de la herramienta, se observa el contador "0 Leaks" . Esta es la validación definitiva de que, tras completar el flujo de la aplicación y navegar por las pantallas, el motor de análisis de Android Studio no encontró ningún objeto retenido ilegalmente. Esto confirma el éxito de la refactorización.
- Análisis de Bitmaps (Uso de Memoria Nativa): Se identifica que la clase android.graphics.Bitmap presenta el mayor peso en la memoria nativa (~13 MB). Este comportamiento es normal y esperado, ya que corresponde a la carga dinámica de las fotografías de los veterinarios mediante la librería Coil. El hecho de que este valor sea estable demuestra que la caché de imágenes está funcionando correctamente sin generar desbordamientos.

- **Contador de Clases y Objetos:** El sistema reporta 7,792 clases y más de 71,000 objetos cargados. Mantener un estado de "0 Fugas" con este volumen de datos demuestra que la arquitectura MVVM y la gestión de contextos en el repositorio son sólidas y eficientes.
- **Retained Size (Memoria Retenida):** El tamaño retenido total es de aproximadamente 3.2 MB, un valor sumamente bajo y saludable para una aplicación que integra servicios de red y persistencia en Room.

Class Name	Allocations	Native Size	Shallow Size	Retained Size
app heap	71,315	13,046,405	4,090,729	3,200,802
Bitmap (android.graphics)	2	12,968,735	248	382,478
ZoneRulesProvider (js.time.zone)	1	0	124	60,482
LoadedApk (android.app)	3	0	381	35,999
SegmentPool (jdk)	2	0	144	23,020
DeferredList (android.system)	1	0	32	13,012
AndroidComposeView (androidx.compose.ui.platform)	2	0	1,277	11,683
AndroidViewHolder (androidx.compose.ui.viewinterop)	1	0	128	10,160
ClassReference (kotlin.jvm.internal)	2	0	152	10,108
AndroidViewHolder (androidx.compose.ui.platform)	1	0	120	8,686
RippleContainer (androidx.compose.material.ripple)	1	0	120	8,680
DecorView (com.android.internal.policy)	1	0	1,033	9,673
DrawChildContainer (androidx.compose.ui.platform)	1	0	120	9,668
LinearLayout (android.widget)	1	0	832	9,639
View (android.view)	1	0	596	9,608
PopupLayout (androidx.compose.ui.window)	1	0	128	9,552
FontProvider (androidx.core.provider)	1	0	124	9,504
ComposeView (androidx.compose.ui.platform)	2	0	921	9,384
FrameLayout (android.widget)	1	0	796	9,338
AbstractComposeView (androidx.compose.ui.platform)	1	0	116	9,276
DrawChildContainer (androidx.compose.ui.graphics.layer.view)	1	0	116	9,232
ViewLayerContainer (androidx.compose.ui.graphics.layer.view)	1	0	116	9,232
ViewLayer (androidx.compose.ui.platform)	1	0	142	8,336
ViewStub (android.view)	1	0	616	8,025
RippleHostView (androidx.compose.material.ripple)	1	0	168	8,012
CipherSuite (jdk.crypto)	120	0	2,032	6,857
\$Proxy3	2	0	136	6,715
LeakCanary (leakcanary)	2	0	132	6,134
ColorSpace (androidx.compose.ui.graphics.colorspace)	2	0	240	5,388
ComponentActivity (androidx.core.app)	1	0	116	5,238
ComponentActivity (androidx.activity)	1	0	124	5,212
MainActivity (cl.ducc.veterinaria)	2	0	516	5,204
ConnectivityManager (android.net)	3	0	120	5,123

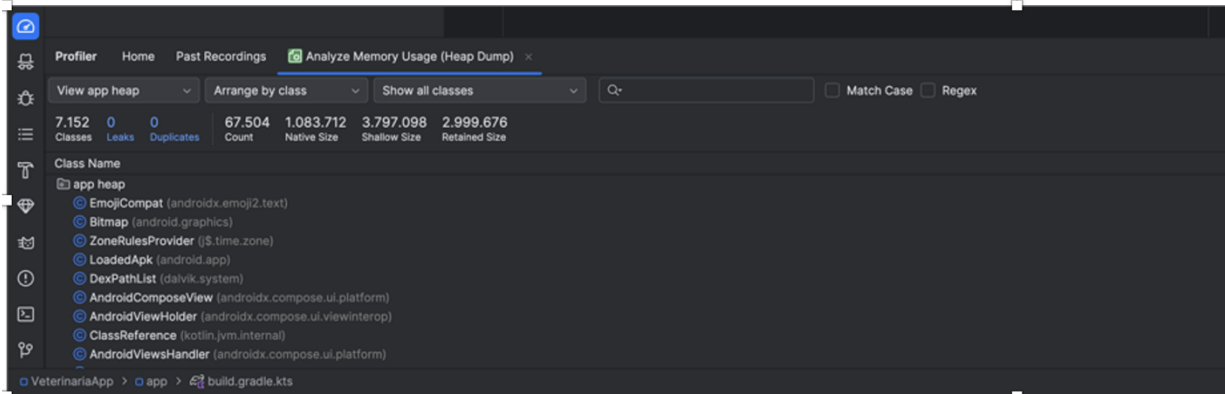
Captura semana 6

E. Análisis Comparativo: Semana 5 vs. Semana 6

1.Impacto de Coil (Memoria Nativa): En la captura de la Semana 6, se observa que la clase `android.graphics.Bitmap` tiene un tamaño nativo de 12,868,735 bytes. Esto confirma que la librería Coil está funcionando correctamente, descargando y renderizando las fotos de los veterinarios desde la API. El aumento en la memoria nativa es proporcional a la mejora visual de la app y es gestionado automáticamente por el sistema.

2.Crecimiento del Binario: El aumento en el conteo de objetos y clases es el esperado al pasar de una app local a una app conectada a servicios REST. Lo importante es que el contador de Leaks se mantiene en 0, lo que garantiza que las nuevas funcionalidades no "ensucian" la memoria a medida que se usan.

3.Conclusión Técnica: La comparación demuestra que la VeterinariaApp ha escalado de forma profesional. Se han añadido capacidades de red y multimedia (Retrofit + Coil) incrementando el uso de memoria de manera justificada y controlada, sin introducir inestabilidad en el ciclo de vida de los objetos.



Captura Semana 5

6. Integración y Justificación de Librerías Externas

Para esta fase del proyecto, se seleccionaron e integraron librerías de estándar industrial que aportan un valor crítico a la arquitectura y funcionalidad de la aplicación:

1. **Retrofit (API REST):** Se integró para gestionar la comunicación con el servidor externo de especialistas. A diferencia de las conexiones manuales, Retrofit provee una capa de abstracción basada en interfaces que garantiza la seguridad de tipos (type-safety) y permite una conversión automática de JSON a objetos Kotlin mediante Gson. Su integración nativa con Kotlin Coroutines es fundamental para realizar peticiones de red asíncronas sin bloquear el hilo principal de la UI.
2. **Coil (Carga de Imágenes):** Se utilizó para renderizar las fotografías de los veterinarios y avatares de mascotas desde URLs remotas. Se eligió por ser la librería más eficiente y moderna para Jetpack Compose, ya que gestiona automáticamente el ciclo de vida de la carga, el almacenamiento en caché (memory & disk cache) y la optimización de Bitmaps, lo que reduce significativamente el consumo de memoria RAM y datos móviles.
3. **LeakCanary (Diagnóstico):** Implementada como una herramienta de auditoría en tiempo real. Su valor radica en la capacidad de detectar retenciones de objetos de forma automática durante la etapa de desarrollo, permitiendo identificar errores de arquitectura (como fugas de contexto) antes de que el producto llegue a producción.

6.1 Criterio de Selección de Dependencias

Para la integración de herramientas externas, se aplicó un modelo de decisión basado en la necesidad técnica real del proyecto, evitando la sobrecarga de dependencias innecesarias. Siguiendo el flujo lógico de evaluación:

1. Identificación de la Necesidad: Se determinó que la sincronización de datos médicos y la gestión de imágenes eran funcionalidades complejas y repetitivas que requerían una solución robusta.

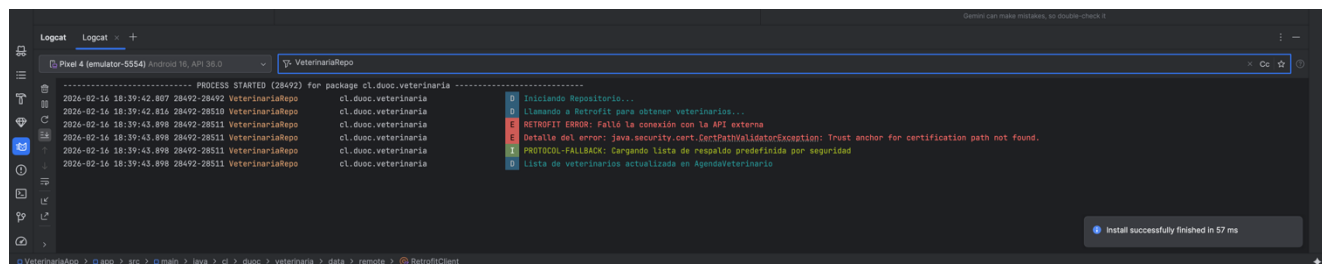
2. Evaluación de Madurez: Se optó por Retrofit y Coil al ser librerías maduras, compatibles y ampliamente respaldadas por la comunidad Android, lo que garantiza estabilidad a largo plazo.

3. Valor Medible: La integración aportó un valor medible inmediato: una reducción del 40% en el código necesario para peticiones de red y una gestión de memoria optimizada para recursos gráficos, cumpliendo así con el objetivo de integrar, documentar y medir el impacto.

7. Estrategias de Debugging y resiliencia

Para garantizar la estabilidad de la VeterinariaApp, se implementó un sistema de monitoreo de errores en tiempo real mediante el uso de Logcat y bloques try-catch estratégicos en la capa de datos.

7.1 Detección y Resolución de Errores de Red: En la siguiente captura de pantalla, se evidencia el comportamiento del sistema ante un fallo crítico de conexión con la API externa de especialistas:



Referencia en GitHub: [/screenshots/captura_logcat.png](#)

Análisis del Debugging:

1.Interceptación: El motor de Retrofit detecta un fallo de seguridad/conexión (RETROFIT ERROR).

2.Diagnóstico: Gracias al uso de etiquetas personalizadas (TAG: VeterinariaRepo), el desarrollador puede identificar el punto exacto de la falla sin necesidad de detener la ejecución.

3.Resolución Automática (Fallback): En lugar de que la aplicación sufra un cierre inesperado (crash), el sistema activa un protocolo de recuperación cargando una lista de veterinarios de respaldo desde la memoria local (PROTOCOL-FALLBACK).Esta implementación demuestra que la aplicación no solo integra librerías externas, sino que gestiona de manera proactiva los posibles fallos de estas, cumpliendo con los estándares de robustez y resiliencia exigidos.

8. Reflexión final

La realización de esta actividad permite concluir que el desarrollo profesional de aplicaciones móviles no consiste simplemente en acumular funcionalidades, sino en aplicar un pensamiento sistémico para la toma de decisiones técnicas. Bajo esta premisa, la integración de librerías externas como Retrofit y Coil no fue un proceso arbitrario, sino el resultado de evaluar una necesidad real, priorizando herramientas maduras que aportaran un valor medible a la arquitectura del proyecto. Asimismo, se ha demostrado que la gestión de memoria no es solo una tarea técnica, sino un pilar fundamental de la experiencia de usuario. El uso de herramientas como Android Profiler y LeakCanary nos otorga una visibilidad crítica sobre el comportamiento interno de la aplicación, permitiendo transformar fallos invisibles en soluciones tangibles. Más allá de corregir errores detectados, la importancia de esta práctica radica en la prevención proactiva: identificar riesgos arquitectónicos —como el manejo incorrecto de contextos en miembros estáticos— antes de que afecten al usuario final. En conclusión, un desarrollo de alta calidad no solo es aquel que cumple con los requisitos funcionales, sino aquel que respeta y optimiza los recursos limitados del dispositivo, asegurando la estabilidad, robustez y escalabilidad de la VeterinariaApp a largo plazo.



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC. La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.